Perforce

June 14, 2026

Ramasamy Kandasamy

1. Perforce (Helix Core)

p4d — The Server

`p4d`	Start the server. `p4d -r <root> -p <port> -d`
`-r <root>`	Server root directory (`P4ROOT`): holds metadata (`db.*`) and versioned depot files.
`-p <port>`	Port to listen on, eg. `1666`.
`-d`	Run as a daemon (background).

```
$ mkdir -p ~/perforce/root
$ p4d -r ~/perforce/root -p 1666 -d
```

Initial Admin Setup

`p4 passwd`	Set the admin (super user) password. *Run this first: with no protections table yet, the connecting user is implicitly super.*
`p4admin`	Admin console — add users, create depots, set protections, etc.

Depots

A **depot** is the top-level container for versioned files on the server — the closest equivalent to a Git "repo".

`local`	*Default* type. Plain versioned files, addressed as `//depot/...`.
`stream`	Files organized under Streams (managed branching / merging).

2. p4 — The Client

Configuration: Env Vars

`P4PORT`	`host:port` of the `p4d` server. Eg: `localhost:1666`.
`P4USER`	Perforce username.
`P4CLIENT`	Name of the workspace (client) to use.

.p4config files

`P4CONFIG` names a config file (eg. `.p4config`). `p4` searches **upward** from the current directory for a file with that name and loads `KEY=VALUE` settings from the first one found. Lets each directory tree pin its own port/user/client.

```
# .p4config
P4PORT=localhost:1666
P4USER=bob
P4CLIENT=bob-ws
```

Loading P4CONFIG

bash	`export P4CONFIG=.p4config` Add to `~/.bashrc` to persist.

PowerShell	`$env:P4CONFIG = ".p4config"`
	Persist: `notepad $PROFILE`.
	`[Environment]::SetEnvironmentVariable( "P4CONFIG", ".p4config", "User")`

p4 info

Shows the current connection / workspace state. Key fields to check:

`Client name`	Workspace in use (`P4CLIENT`).
`Client root`	Local directory the client maps to.
`Current directory`	`cwd` that `p4` sees.

Scope of Commands

- Commands like `p4 status` (no path args) only act on the **subtree rooted at the current directory**.
- The current directory must be **inside the client root**. If `cwd` is outside the client root, these commands find nothing / fail.

3. p4 — Working with Files

Note: `...` is a recursive wildcard matching all files under a path (eg. `p4 revert ...` = everything under cwd).

Sync

`p4 sync [path...]`

Updates the workspace to match the depot, per the client view (like `git pull` for files). With no path, acts on the subtree under cwd.

`path#<rev>`	Sync to a specific revision.
`path@<cl/label>`	Sync to the state as of a changelist, label, or date.
`-f`	Force: re-transfer files even if already at that revision (eg. to repair local edits made outside p4).
`-n`	Preview only — no files transferred.
`-k`	Update the have-list (bookkeeping) without transferring files.
`-p`	Populate files without updating the have-list.

Open for Change

`p4 edit <file>`	Open an existing depot file for edit (checkout).
`p4 add <file>`	Mark a new (untracked) file for add.
`-c <cl>`	Open in pending changelist `<cl>` instead of `default`. Applies to `edit`, `add`, `revert`, `reopen`, `shelve`, `unshelve`, `submit`.

Reconcile

`p4 reconcile [-a] [-e] [-d] [-c <cl>] [path...]`

Detects changes made outside `p4` and opens files accordingly. No flags = all of `-a -e -d`.

`-a`	Open files added on disk `for add`.
`-e`	Open modified files `for edit`.
`-d`	Open missing files `for delete`.

Revert / Reopen

`p4 revert [-a] [-c <cl>] <file>`	Discard pending changes, restore depot version.
`-a`	Only revert files *unchanged* from depot (safe revert).
`p4 reopen -c <cl> <file>`	Move opened file(s) to a different pending changelist.

Shelve / Unshelve

`p4 shelve -c <cl> [file]`	Save files opened in `<cl>` to the server without submitting.
`-d`	With `shelve`: discard the shelved file(s) from `<cl>`.
`p4 unshelve -s <cl> [-c <cl2>] [file]`	Restore shelved files from `<cl>` (`-s`) into changelist `<cl2>` (`-c`, default `default`).

Submit

`p4 submit -c <cl>`	Submit pending changelist `<cl>` to the depot.

Inspecting

`p4 opened [-c <cl>] [-u <user>]`	List opened (checked-out) files. `-c`: by changelist. `-u`: by user.
`p4 filelog <file>`	Show a file's revision history.
`p4 changes [-u <user>] [-m <n>] [-s <status>]`	List changelists. `-u`: by user. `-m <n>`: limit to `n`. `-s`: `pending` / `submitted` / `shelved`.